CSCI-C 212 Exam 2 Fall 2026 (80 points)
Nov 2, 2026

C212 Exam 2 Rubric

1. (20 points) Design the `Map<String, List<Integer>> kLowestGrades(Map<String, List<Integer>> grades, int k)` method that, when given a map of names to numeric grades *grades* and an integer $k \geq 1$, returns a lexicographically-ordered map of the lowest k grades, if they exist, from each of the names. Each list should be sorted in increasing order.

   For example, suppose we define *grades* as follows:

   $$\begin{aligned}
   \texttt{"Ransom"} &\mapsto [70, 45] \\
   \texttt{"Sunny"} &\mapsto [101, 22, 22, 23, 84] \\
   \texttt{"Matthew"} &\mapsto [98] \\
   \texttt{"Peter"} &\mapsto [] \\
   \texttt{"Elliot"} &\mapsto [100, 101, 45] \\
   \texttt{"Carley"} &\mapsto [67, 82, 103] \\
   \texttt{"Laura"} &\mapsto [100, 95]
   \end{aligned}$$

   If $k = 2$, then `kLowestGrades` returns:

   $$\begin{aligned}
   \texttt{"Carley"} &\mapsto [67, 82] \\
   \texttt{"Elliot"} &\mapsto [45, 100] \\
   \texttt{"Laura"} &\mapsto [95, 100] \\
   \texttt{"Matthew"} &\mapsto [98] \\
   \texttt{"Peter"} &\mapsto [] \\
   \texttt{"Ransom"} &\mapsto [45, 70] \\
   \texttt{"Sunny"} &\mapsto [22, 22]
   \end{aligned}$$

   Huge *hint: use a* `PriorityQueue!`

THE SKELETON CODE IS ON THE NEXT PAGE.
WRITE YOUR SOLUTION THERE.

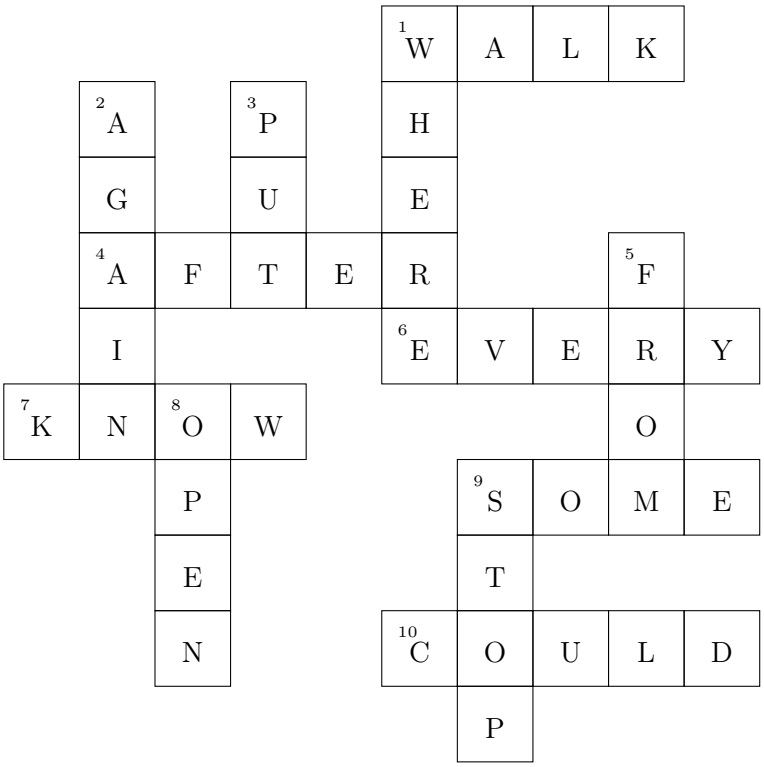
Solution:

```
static Map<String, List<Integer>> kLowestGrades(Map<String, List<Integer>> grades,
                                                int k) {
  Map<String, List<Integer>> result = new TreeMap<>();
  for (String name : grades.keySet()) {
    Queue<Integer> pq = new PriorityQueue<>();
    for (int v : grades.get(name)) {
      pq.add(v);
    }
    List<Integer> curr = new ArrayList<>();
    int i = 0;
    while (!pq.isEmpty() && i < k) {
      curr.add(pq.poll());
      i++;
    }
    result.put(name, curr);
  }
  return result;
}
```

Rubric (20 points):

- **3 points:** Creates a result `TreeMap`.
- **3 points:** Iterates over every name (+1.5) and accesses its corresponding list of grades. (+1.5)
- **4 points:** Places all grades, including duplicates, into a priority queue.
- **2 points:** Creates a separate result list for each name.
- **3 points:** Stops after k grades or when no grades remain (+0.5 for each condition); handles empty lists and lists with fewer than k grades. (+2)
- **3 points:** Removes the smallest remaining grades and appends them in increasing order, updating the count correctly.
- **2 points:** Associates each name with its result list, including empty lists, and returns the completed map.

2. Consider a crossword puzzle defined as a `String[][]`, where each cell is either a *one-string*, i.e., a string of one character, or `null` to indicate that the cell is not used by the crossword.



The same filled crossword can be represented as a `String[][]`:

```
String[][] crossword = {
    {null, null, null, null, null, "W",  "A",  "L",  "K",  null},
    {null, "A",  null, "P",  null, "H",  null, null, null, null},
    {null, "G",  null, "U",  null, "E",  null, null, null, null},
    {null, "A",  "F",  "T",  "E",  "R",  null, null, "F",  null},
    {null, "I",  null, null, null, "E",  "V",  "E",  "R",  "Y" },
    {"K",  "N",  "O",  "W",  null, null, null, null, "O",  null},
    {null, null, "P",  null, null, null, "S",  "O",  "M",  "E" },
    {null, null, "E",  null, null, null, "T",  null, null, null},
    {null, null, "N",  null, null, "C",  "O",  "U",  "L",  "D" },
    {null, null, null, null, null, null, "P",  null, null, null}
};
```

In this question, you will design a `Crossword` class to represent crossword puzzles and search for words in the puzzle.

(Fun fact: this question is inspired by my AP Computer Science A exam from 2016. Yes, that exam was from hell... I hope this one is not!)

CONTINUE ONTO THE NEXT PAGE.

(a) (6 points) Design the `Crossword` class, whose constructor receives the `String[][]` board and copies it into a *referentially final* instance variable array. **Do not just reassign the reference!**

Solution:

```
class Crossword {
  private final String[][] CROSSWORD;

  Crossword(String[][] board) {
    this.CROSSWORD = new String[board.length][board[0].length];
    for (int r = 0; r < this.CROSSWORD.length; r++) {
      for (int c = 0; c < this.CROSSWORD[r].length; c++) {
        this.CROSSWORD[r][c] = board[r][c];
      }
    }
  }
}
```

Rubric (6 points):

- **1 point:** Constructor accepts a `String[][]`.
- **1 point:** Declares a `private final String[][]` instance variable. +0.5 for each keyword. No points are awarded if it's the wrong type.
- **2 points:** Allocates an array of the correct dimensions.
- **2 points:** Copies every cell, including `null`, into the corresponding position in the new array.

(b) (4 points) Certain cells in a crossword puzzle are *labeled*, meaning they start at least one word. Cells that are *not* labeled cannot start a word. There are two rules for whether a cell is labeled:

1. The cell is in bounds and non-`null`.
2. The cell starts either an "across word" or a "down word."

Design the `boolean isLabeled(int r, int c)` method that returns whether the cell at (r, c) is labeled according to the above rules.

We provide two helper methods: `boolean startsAcross(int r, int c)` and `boolean startsDown(int r, int c)` that determine if the cell at (r, c) starts a word going across or going down, respectively.

We also provide the helper method `boolean inBounds(int r, int c)`, which returns whether the cell (r, c) is within the bounds of the crossword.

Your implementation must use these three methods correctly or you will not receive credit.

THE SKELETON CODE IS ON THE NEXT PAGE.
WRITE YOUR SOLUTION THERE.

Solution:

```
boolean isLabeled(int r, int c) {
  return this.inBounds(r, c)
      && this.CROSSWORD[r][c] != null
      && (this.startsAcross(r, c) || this.startsDown(r, c));
}
```

Rubric (4 points):

- **1 point:** Uses `inBounds(r, c)` to guard array access and subsequent helper calls. Award the point even if this check is not in the correct order (e.g., the `null` check comes first).
- **1 point:** Checks that the cell is non-`null`.
- **1 point:** Combines `startsAcross(r, c)` and `startsDown(r, c)` with logical OR.
- **1 point:** Returns true exactly when the bounds, non-`null`, and word-start conditions all hold.

As stated in the question, award 0 points for this part if the three required helper methods are not used correctly.

(c) (20 points) Design the `List<String> findWords(int r, int c)` method that finds the words when starting at the labeled cell (r, c). Note that intersecting into another labeled word does not start a new word. Below is the pseudocode to write this algorithm, which extends out the current string across and down as long as the cells exist.

To exemplify the algorithm, calling `findWords(0, 5)` on the above crossword returns `["WALK", "WHERE"]`. Importantly, the returned list does *not* include `"EVERY"`.

Your solution must use the three helper methods provided in part (b) correctly or you will not receive credit.

```
findWords(r, c):
  words <- []
  if startsAcross(r, c) then
    c' <- c
    str <- ""
    while (r, c') is in bounds and crossword at (r, c') is not null do
      str <- str + crossword at (r, c')
      increment c'
    add str to words

  if startsDown(r, c) then
    r' <- r
    str <- ""
    while (r', c) is in bounds and crossword at (r', c) is not null do
      str <- str + crossword at (r', c)
      increment r'
    add str to words

  return words
```

THE SKELETON CODE IS ON THE NEXT PAGE.
WRITE YOUR SOLUTION THERE.

Solution:

```
List<String> findWords(int r, int c) {
  List<String> words = new ArrayList<>();
  if (this.startsAcross(r, c)) {
    int col = c;
    String word = "";
    while (this.inBounds(r, col) && this.CROSSWORD[r][col] != null) {
      word += this.CROSSWORD[r][col];
      col++;
    }
    words.add(word);
  }
  if (this.startsDown(r, c)) {
    int row = r;
    String word = "";
    while (this.inBounds(row, c) && this.CROSSWORD[row][c] != null) {
      word += this.CROSSWORD[row][c];
      row++;
    }
    words.add(word);
  }
  return words;
}
```

Rubric (20 points):

- **1 point:** Initializes a list to hold the words.
- **2 points:** Uses `startsAcross(r, c)` to conditionally perform the across scan.
- **2 points:** Initializes the across column to `c` and its accumulator to the empty string.
- **3 points:** Continues the across scan only while `inBounds(r, col)` holds and the current cell is non-`null`, checking bounds before array access.
- **2 points:** Appends the across cell and increments the column while keeping the row fixed.
- **1 point:** Adds the completed across word exactly once, after the loop.
- **2 points:** Uses a separate `startsDown(r, c)` conditional so that both words can be found.
- **2 points:** Initializes the down row to `r` and a fresh empty-string accumulator.
- **3 points:** Continues the down scan only while `inBounds(row, c)` holds and the current cell is non-`null`, checking bounds before array access.
- **1 point:** Appends the down cell and increments the row while keeping the column fixed.
- **1 point:** Adds the completed down word exactly once and returns the list.

As stated in the question, award 0 points for this part if the three required helper methods are not used correctly.

CONTINUE ONTO THE NEXT PAGE.

(d) (14 points) Overload the `findWords` method to return *all* of the words in the crossword. **Your solution must use `isLabeled` correctly or you will not receive credit.**

Solution:

```
List<String> findWords() {
  List<String> words = new ArrayList<>();
  for (int r = 0; r < this.CROSSWORD.length; r++) {
    for (int c = 0; c < this.CROSSWORD[r].length; c++) {
      if (this.isLabeled(r, c)) {
        words.addAll(this.findWords(r, c));
      }
    }
  }
  return words;
}
```

Rubric (14 points):

- **2 points:** Initializes a list to hold all words.
- **3 points:** Iterates over all rows with correct bounds.
- **3 points:** Iterates over all columns of each row with correct bounds.
- **2 points:** Uses `isLabeled(r, c)` to process only labeled cells.
- **3 points:** Calls `findWords(r, c)` and adds all returned words to the result (e.g., using `addAll`).
- **1 point:** Returns the completed list.

As stated in the question, award 0 points for this part if `isLabeled` is not used correctly. Grade calls to earlier methods assuming those methods work as specified.

(e) (8 points) Design the `List<String> findWordsOfLength(int n)` method that returns all words that are exactly of length n. Part of the code has been provided. **You must use only the Stream API.**

Solution:

```
List<String> findWordsOfLength(int n) {
  return this.findWords().stream()
       .filter(word -> word.length() == n)
       .toList();
}
```

Rubric (8 points):

- **3 points:** Uses `filter` with a predicate on each word.
- **3 points:** Keeps exactly the words satisfying `word.length() == n`.
- **2 points:** Uses a terminal operation that produces a `List<String>` (e.g., `toList()` or `collect(Collectors.toList())`).

The solution must use only the Stream API, as required by the question; award 0 points for a loop-based replacement.

(f) (8 points) Override the `public boolean equals(Object o)` method to return whether two `Crossword` objects are equal. They are equal if and only if their underlying arrays are equal.

Solution:

```
@Override
public boolean equals(Object o) {
  if (!(o instanceof Crossword)) {
    return false;
  } else {
    Crossword other = (Crossword) o;
    return Arrays.deepEquals(this.CROSSWORD, other.CROSSWORD);
  }
}
```

Rubric (8 points):

- **2 points:** Checks whether `o` is a `Crossword`, rejecting `null` and objects of other types.
- **1 point:** Returns false when the type check fails.
- **1 point:** Safely obtains a `Crossword` reference by casting or pattern matching.
- **4 points:** Compares the full contents of the two-dimensional arrays, including `null` cells, and returns that result (e.g., `Arrays.deepEquals`). Reference equality or outer-array-only comparison does not earn these points. Using `Arrays.equals` or calling `equals` on an array does not earn the points.